A Simulation-based Online Evolutionary Algorithm for Combat in StarCraft II

Yongfeng Liu*, Changhe Li*, Sanyou Zeng†
*School of Automation, China University of Geosciences, Wuhan 430074, China
Hubei Key Laboratory of Advanced Control and Intelligent Automation for Complex Systems, Wuhan 430074, China
†School of Mechanical Engineering and Electronic Information, China University of Geosciences, Wuhan 430074, China
Email: 20131002682@cug.edu.cn, changhe.lw@gmail.com, sanyouzeng@gmail.com

***Abstract*—Real-time strategy (RTS) games have become one of the hotspots in the field of artificial intelligence research due to the large search space, long-term planning, and real-time constraint. In view of the fact that most of the current research on solving the RTS combat problem uses offline training methods, and these methods may need a lot of training time, this work uses an evolutionary algorithm (EA) to directly optimize the units' action sequences online. In order to solve the problems of repetitive actions caused by the neutral area and the size correlation caused by cooperation between units, three search operators are proposed. It is shown that this method can effectively solve the combat problem in RTS games.**

***Index Terms*—real-time strategy, evolutionary algorithm, online optimization**

I. Introduction

RTS games are a type of video game where two or more players each controlling a group of units, try to defeat the enemy through the collection, construction, and battles in real-time. StarCraft II (SC2) is one of the representatives. In this game, combat is the most important part – regardless of the player's ability to collect and build, the winner will ultimately be determined through battles. The so-called combat problem is that in a limited area, the two sides compete to destroy the other side by controlling their units to complete sequences of commands. Its huge action space, long-term planning, and real-time constraint have brought challenges to the current artificial intelligence (AI) research.

Among all the methods, reinforcement learning (RL) is the most widely used. The most notable work should be the one carried out by Google DeepMind team after 2017 [1]–[3]. AlphaStar finally defeated professional human players based on the multi-agent RL [3]. The optimization methods are also used, but most of them are offline methods. For example, Liu et al. used a genetic algorithm (GA) to optimize the influence map (IM) and potential field (PF), which then performed as the decision model [4]. Based on some predefined tactics, Hsu and Chen used a GA to optimize the selection thresholds of designed tactics selector [5]. Meanwhile, there is also a small amount of work based on online optimization. Churchill and Buro used a greedy search to optimize the sequence of some predefined tactics [6]. Inspired by [6] and [7], Wang et al. replaced greedy search with a population-based method and obtains better results [8].

Given that learning-based and offline optimization methods require a lot of time for training, and existing online optimization methods need low-level tactics, this work takes advantage of online optimization which doesn't need training, and attempts to use it to directly optimize the sequences of actions to avoid writing the underlying tactics.

II. Methods

The following content will explain the necessary parts of the algorithm such as the solution representation, evaluation method, solution selection, deployment, and three search operators.

A. Solution Representation and Deployment

A solution contains the sequence of actions to be performed by all units in the next period of time. Suppose we have M units, and each unit has C actions, then the i-th unit's j-th action can be represented as a_{ij}, $i \in \{1, 2, \ldots, M\}$, $j \in \{1, 2, \ldots, C\}$, then a solution s can be representated as

$$s = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1L} \\ a_{21} & a_{22} & \cdots & a_{2L} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M1} & a_{M2} & \cdots & a_{ML} \end{pmatrix}$$

where a is a 2-tuple, i.e. $a = (actionId, targetPoint)$. $actionId$ represents the action to be deployed; $targetPoint$ determines the target coordinates of the action.

For now, we just consider move and attack actions. The way to deploy them in a solution is shown in Algorithm 1.

Algorithm 1 Solution Deployment

```
function DEPLOY(Genome g, State s)
    for u in s.myUnits do
        if u is idle then
            if u has finished moving and there are enemies within attack range of u then
                attack the nearest enemy;
            else
                a ← the next action for u from g;
                move u to a.targetPoint;
```

978-1-7281-6517-2/20/$31.00 ©2020 IEEE
DOI 10.1109/IHMSC49165.2020.10104

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 31,2026 at 06:16:39 UTC from IEEE Xplore. Restrictions apply.

B. Solution Evaluation

A solution is evaluated in this way: input the current state and action sequences from the solution into a simulator, then forward the simulator until the game loops reach simulation length l^{sim} and evaluate that solution based on simulation data (Algorithm 2). Since we try to directly optimize the actions, the simulation needs to be as consistent as possible with the actual game, so we call other SC2 game processes for simulation by using the debug interface of SC2API. The deployment of the action in the simulator is the same as in the actual game as described in Algorithm 1.

After the simulation, We choose a 3-objectives evaluation model to reflect the quality of a solution:

$$\max f_1 = -(\sum_{i=1}^{M}(h_i^{init} + s_i^{init} - h_i^{end} - s_i^{end}) \quad (1)$$

$$\max f_2 = \sum_{i=M+1}^{M+N}(h_i^{init} + s_i^{init} - h_i^{end} - s_i^{end}) \quad (2)$$

$$\max f_3 = \begin{cases} -l^{end} & \text{if win} \\ -l^{sim} & \text{else} \end{cases} \quad (3)$$

where N means that the enemy team has N units: $M+1, \ldots, M+N$; h_i^{init} and s_i^{init} are the health points (HP) and shield of unit i at the beginning of the simulation, respectively; h_i^{end} and s_i^{end} are the final HP and shield of unit i, respectively; l^{end} is the loop at which the simulation is ended because one team has been eliminated. Thus f_1 and f_2 are the damage of both teams during the simulation, respectively, and f_3 is the loops needed to finish the game. f_3 only works when our team can win during the simulation. Optimizing it can reduce a lot of meaningless actions at the end of the game.

Algorithm 2 Evaluation

Data: $simLength$: simulation loops, i.e. l^{sim}
function EVALUATE(Genome g, State s, Int $simLength$)
  sim.SetState(s)
  for $loop = 0$ to $simLength$ **do**
    Deploy(g);
    $g.simData$.Record(sim.GetState());
    sim.Forward();
  use $g.simData$ to set $g.objs$;

C. Solution Selection

The selection is based on the first two objectives:

$$s' = \arg\max_s (f_1 + f_2) \quad (4)$$

where s' is the solution to be deployed. The equation 4 is the difference between the damage taken by both sides. It can reflect the quality of offense and defense of a solution at the same time.

D. Search Operators

The combat problem has a large search space. Besides, the main purpose of the player is to wipe out the enemy as much as possible. And compared to the vast map, searching escape strategy is relatively simple, so how to effectively organize the offense is more difficult and important. So we add three attacking search operators.

1) Priori Solution Insertion: The purpose is to roughly reduce the pressure of the algorithm to deal with the correlation between units and guide the population to quickly find the enemy (Algorithm 3).

Algorithm 3 Priori Solution Insertion

function PRIOIRINSERT(Position $enemyPos$, State s)
  $M \leftarrow$ number of my units;
  $C \leftarrow$ action sequence length;
  $g \leftarrow$ new Genome(M, C);
  for $x = 0$ to M **do**
    $p \leftarrow$ position around $enemyPos$;
    Fill($g[x]$, $\{ATTACK, p\}$);
  return g;

2) Assemble Operator: Considering that we can know get statistics of each unit through the simulation, and there is certain independence between different units, we consider putting together those excellent units from different solutions to form a potentially better solution (Algorithm 4).

Algorithm 4 Assemble Operator

function ASSEMBLE(Genome[] pop, State s)
  $M \leftarrow$ number of my units;
  $g \leftarrow$ new Genome($M, 0$);
  for $x = 0$ to M **do**
    $bestUnitG \leftarrow$ the genome in which my x-th unit has the highest damage in ;
    $g[x] \leftarrow bestUnitG[x]$;
  return g;

3) Fix Operator: In order to alleviate the meaningless movement of our units (for example, some units run too far away from the crossfire area), we designed this fix operator. Based on the simulation data, it moves the meaningless action targets around the enemies (Algorithm 5).

E. Algorithm Structure

The algorithm will run at a regular interval during the game. At each run, a set of solutions is generated. Those solutions are evaluated by the simulators, and the optimal solution set is obtained through multiple iterations of evolution. Finally, one of the solutions is selected and deployed (Algorithm 6). The basic optimization method is a differential evolution (DE) algorithm [9] since it can effectively optimize the real number coordinates. And the observation and control of SC2 is provided by StarCraft II C++ API (SC2API) [1] .

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 31,2026 at 06:16:39 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5 Fix Operator

function FIX(Genome g, State s)
  $c \leftarrow g$;
  $M \leftarrow$ number of my units;
  $C \leftarrow$ action sequence length;
  for $i = 0$ to M **do**
    for $i = 0$ in C **do**
      if $g[i][j].actionId$=$ATTACK$ and it didn't attack any enemy **then**
        $t \leftarrow$ the loop when $g[i][j]$ is finished;
        $enemy_locs \leftarrow$enemy locations at t;
        move $g[i][j].targetPoint$ around the nearest location in $enemy_locs$;
  return c;

Algorithm 6 Online Evolution

Input: $popSize$: size of population
Input: $maxGeneration$: termination condition
Input: $fixInterval$: the generation interval to use fix opeartor
Input: s: current game state
Input: $simLength$: simulation loops, i.e. l^{sim}
Data: $offspring$: offspring of current population
Output: genomes on Pareto front in final population

$pop \leftarrow$ Genome[$popSize$];
Init(pop, s);
while $currentGeneration < maxGeneration$ **do**
  if $currentGeneration\%fixInterval = 0$ **then**
    for Genome g in pop **do**
      $offspring$ += Fix(g, s);
  else
    for Genome g in pop **do**
      $offspring$ += DEReproduce(g, RandomEntry(pop), RandomEntry(pop));
  RandomEntry($offspring$) = Assemble(pop, s);
  pop.Insert($offspring$);
  Select(pop, $popSize$, $simLength$, s);
return Pareto-optimal set in pop;

Algorithm 7 Initialization and Selection

function INIT(Genome[] pop, State s)
  for $x = 0$ to $pop.size$ **do**
    if $x < s.enemies.size$ **then**
      $pop[x] \leftarrow$ PrioirInsert($s.enemies[x].pos$, s);
    else
      $pop[x] \leftarrow$ RandomInit(s);

function SELECT(Genome[] pop, Int $popSize$, Int $simLength$, State s)
  for Genome g in pop **do**
    Evaluate(g, s, $simLength$);
  DominanceSort(pop);
  pop.Resize($popSize$);

III. EXPERIMENTS AND RESULTS

A. Map Design

We designed three maps, the first and second are static maps where the enemies are unmovable. And the third is a dynamic map. All the maps in this paper are for 2-player battles. All map sizes are 32*32 without any obstacles.

Map 1 (Figure 1a): Our team has three Thors and one Marine. The enemy team has 7 Auto-turrets. Some toxic areas are located in the middle of the map and will harm our units. The focus of this scenario is to test whether the algorithm can find and maintain reasonable tactical positions for different units. In this map, our Thors whose fire range is longer than the Auto-turrets needs to avoid the toxic areas and attack the Auto-turrets. The Marine shouldn't attack since its fire range is shorter than the Auto-turrets.

Map 2 (Figure 1b): The enemy units are Auto-turrets. We have some Marines. This map is used to test the cooperation ability between units. Single Marine is vulnerable. So if our team wants to defeat the enemy, all units must attack the same Auto-turret at the same time.

Map 3 (Figure 1c): The purpose of this map is to test a classic tactic – kiting. We have some Marines, and the enemy is several Zealots. Zealot can only engage in melee attacks, while Marine can attack Zealot from a distance. The tactic of Zealots is to attack the nearest enemy. The best tactic for Marines is to move away from Zealots and attack when it can.

B. Algorithm Setup

In this work, the action sequence length C is set to 20 for each unit. The interval between two runs during a game is set to 200 loops and the simulation length l^{sim} is set to 250 loops. The reason why the simulation length is slightly longer than the running interval is to ensure that the actions at the end of the sequences can be completed. It also provides a forward-looking ability for the algorithm. The DE's crossover rate is 0.9 and the mutation rate is 0.1. The population size is set to 50 and the termination condition is 100 generations.

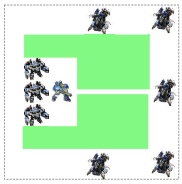
C. Results

1) Intuitive Results: In Map 1, our Thors can approach the enemies along a narrow security area and stand outside the enemies' fire range to attack them. The only Marine roams outside the enemies' fire range. In Map 2, our Marines usually start to focus on the nearest Auto-turret, then destroy the remaining enemies one by one in the same way. In Map 3, a few Marines attract enemy Zealots. The remaining Marines follow and attack them at the same time.

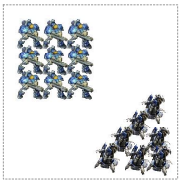
2) Analysis: The following analysis is mainly on Map 3 since it is the most difficult one in our experiment.

Figure 2a and 2b show the effect on the game result of different operator settings . It can be seen from Figure 2 that all search operators have improved the performance of the original DE-based algorithm (n). From the perspective of offensive efficiency, Figure 2b shows that paf takes roughly half of the time of n; from the perspective of defensive efficiency, n suffered the least average casualty.

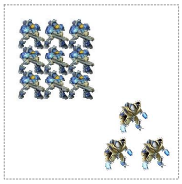
Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 31,2026 at 06:16:39 UTC from IEEE Xplore. Restrictions apply.



(a) Map 1: Auto-turrets vs Thors and a Marine. Toxic areas are indicated in green

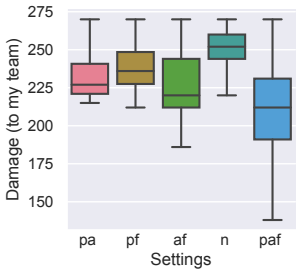


(b) Map 2: Auto-turrets vs Marines

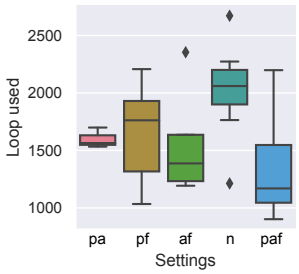


(c) Map 3: Zealots vs Marines

Fig. 1: Experimental maps



(a) Damage to my team



(b) Game loops used

Fig. 2: Game results under different settings, where p, a, and f represent priori insertion, assemble, and fix operators, respectively

Figure 3 shows the distribution of the population under different settings. From the damage perspective, it can be seen that the value of paf is twice that of n. From the HP loss perspective, paf is a little worse than n, but it is not significant.

In summary, we can draw that the improved algorithm is more offensive than the original algorithm, besides, the efficient attack also reduced our casualties.

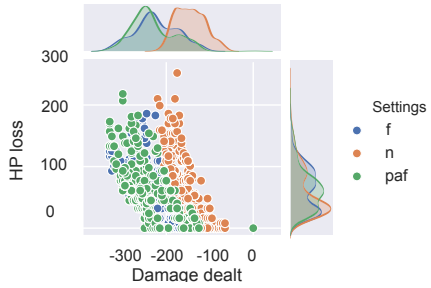


Fig. 3: Population distributions on f_1 and f_2 in different settings, from the first algorithm run at the beginning of the game. Only individuals in Pareto front are showed. p, a, and f represent priori, assemble, and fix operators, respectly.

IV. Conclusion

This work demonstrates the effectiveness of the EA in solving the combat problem of SC2 by optimizing the action sequences, followed by the search operators used to effectively improve algorithm's performance in decreasing redundant actions and speeding up convergence. And there is no need to train and add existing tactics as portfolio.

In the future, we may consider modeling the opponent or using robust optimization to handling the uncertainty of the opponent's strategy thus improve the robustness of the solution. In addition, we consider using a surrogate model to estimate the simulation results to reduce the evaluation time consumption.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 61673355, in part by the Fundamental Research Funds for the Central Universities, China University of Geosciences(Wuhan) under Grant CUGGC02, in part by the Hubei Provincial Natural Science Foundation of China under Grant 2015CFA010, in part by the 111 project under Grant B17040.

References

[1] O. Vinyals, T. Ewalds, S. Bartunov, P. Georgiev, A. S. Vezhnevets, M. Yeo, A. Makhzani, H. Küttler, J. Agapiou, J. Schrittwieser *et al.*, "Starcraft ii: A new challenge for reinforcement learning," *arXiv preprint arXiv:1708.04782*, 2017.

[2] O. Vinyals, I. Babuschkin, J. Chung, M. Mathieu, M. Jaderberg, W. Czarnecki, A. Dudzik, A. Huang, P. Georgiev, R. Powell, T. Ewalds, D. Horgan, M. Kroiss, I. Danihelka *et al.*, "AlphaStar: Mastering the Real-Time Strategy Game StarCraft II," https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/, 2019.

[3] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[4] S. Liu, S. J. Louis, and C. Ballinger, "Evolving effective micro behaviors in rts game," in *2014 IEEE Conference on Computational Intelligence and Games*. IEEE, 2014, pp. 1–8.

[5] W.-L. Hsu and Y.-p. Chen, "Learning to select actions in starcraft with genetic algorithms," in *2016 Conference on Technologies and Applications of Artificial Intelligence (TAAI)*. IEEE, 2016, pp. 270–277.

[6] D. Churchill and M. Buro, "Portfolio greedy search and simulation for large-scale combat in starcraft," in *2013 IEEE Conference on Computational Inteligence in Games (CIG)*. IEEE, 2013, pp. 1–8.

[7] N. Justesen, T. Mahlmann, and J. Togelius, "Online evolution for multi-action adversarial games," in *European Conference on the Applications of Evolutionary Computation*. Springer, 2016, pp. 590–603.

[8] C. Wang, P. Chen, Y. Li, C. Holmgård, and J. Togelius, "Portfolio online evolution in starcraft," in *Twelfth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2016.

[9] R. Storn and K. Price, "Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces," *Journal of global optimization*, vol. 11, no. 4, pp. 341–359, 1997.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 31,2026 at 06:16:39 UTC from IEEE Xplore. Restrictions apply.